



OOP in Java

Slide 8 - Constructor, Static, Wrapper Class and ArrayList

Ishan Shrestha



In Academic Collaboration with



A • P • U
ASIA PACIFIC UNIVERSITY
OF TECHNOLOGY & INNOVATION

What is a Constructor?

- **Definition:** A constructor is a special method used to initialize objects.
- **Key Features:**
 - Has the same name as the class.
 - No return type (not even void).
 - Automatically called when an object is created.

Default Constructor

► Explanation:

- Provided by Java if no constructor is defined.
- Initializes instance variables to default values.

```
Main.java x
1  class Car { 2 usages
2      String brand; 1 usage
3      int year; 1 usage
4  }
5
6  public class Main {
7      public static void main(String[] args) {
8          Car myCar = new Car();
9          System.out.println(myCar.brand); // Output: null
10         System.out.println(myCar.year); // Output: 0
11     }
12 }
```

Parameterized Constructor

- Explanation:
 - Allows passing arguments to initialize the object with specific values

```
Main.java x
1  class Car { 2 usages
2      String brand; 2 usages
3      int year; 2 usages
4
5      // Parameterized Constructor
6      Car(String b, int y) { 1 usage
7          brand = b;
8          year = y;
9      }
10 }
11
12 public class Main {
13     public static void main(String[] args) {
14         Car myCar = new Car(b: "Toyota", y: 2022);
15         System.out.println(myCar.brand); // Output: Toyota
16         System.out.println(myCar.year); // Output: 2022
17     }
18 }
```

Difference: Constructor vs. Method

Feature	Constructor	Method
Purpose	Initialize objects	Perform actions
Name	Same as the class	Can be any valid name
Return Type	None	Must have a return type
Invocation	Called automatically	Called explicitly

Interactive Activity

- ▶ Write a class Car with:
 - ▶ Instance variables brand and year.
 - ▶ A parameterized constructor to initialize these variables.
 - ▶ A method to print the details of the car.

What is the Static Keyword?

- ▶ **Definition:** static is a keyword in Java used for defining class-level members that belong to the class rather than any specific object.
- ▶ **Key Features:**
 - ▶ Shared across all instances of the class.
 - ▶ Accessible without creating an object.

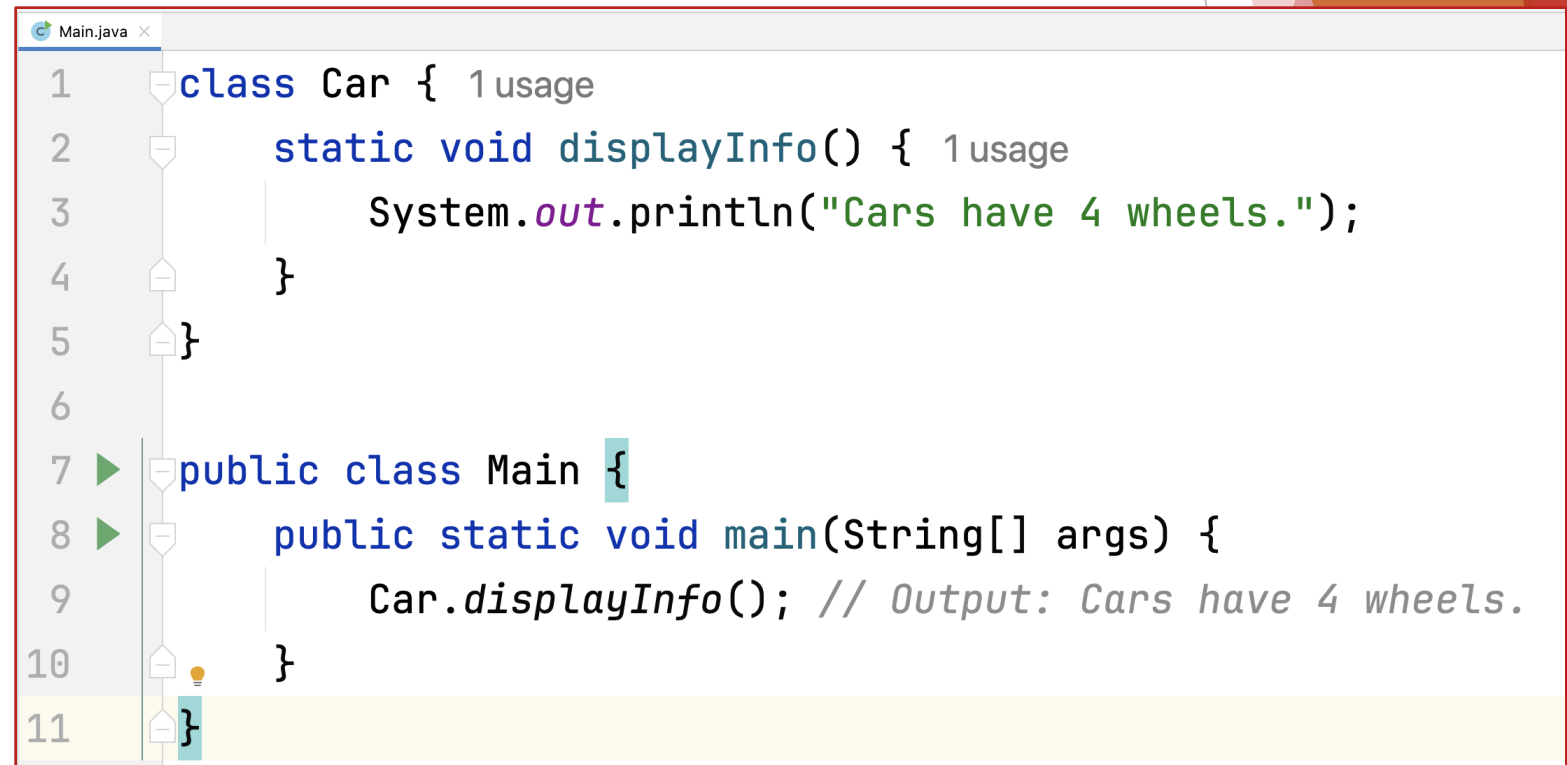
Static Variables

- ▶ A static variable is shared among all objects of the class.
- ▶ It is initialized only once at the class loading time.

```
Main.java x
1  class Car { 1 usage
2      static int wheels = 4; // Static variable 1 usage
3      String brand; no usages
4  }
5
6  public class Main {
7      public static void main(String[] args) {
8          System.out.println(Car.wheels); // Output: 4
9      }
10 }
```


Static Methods

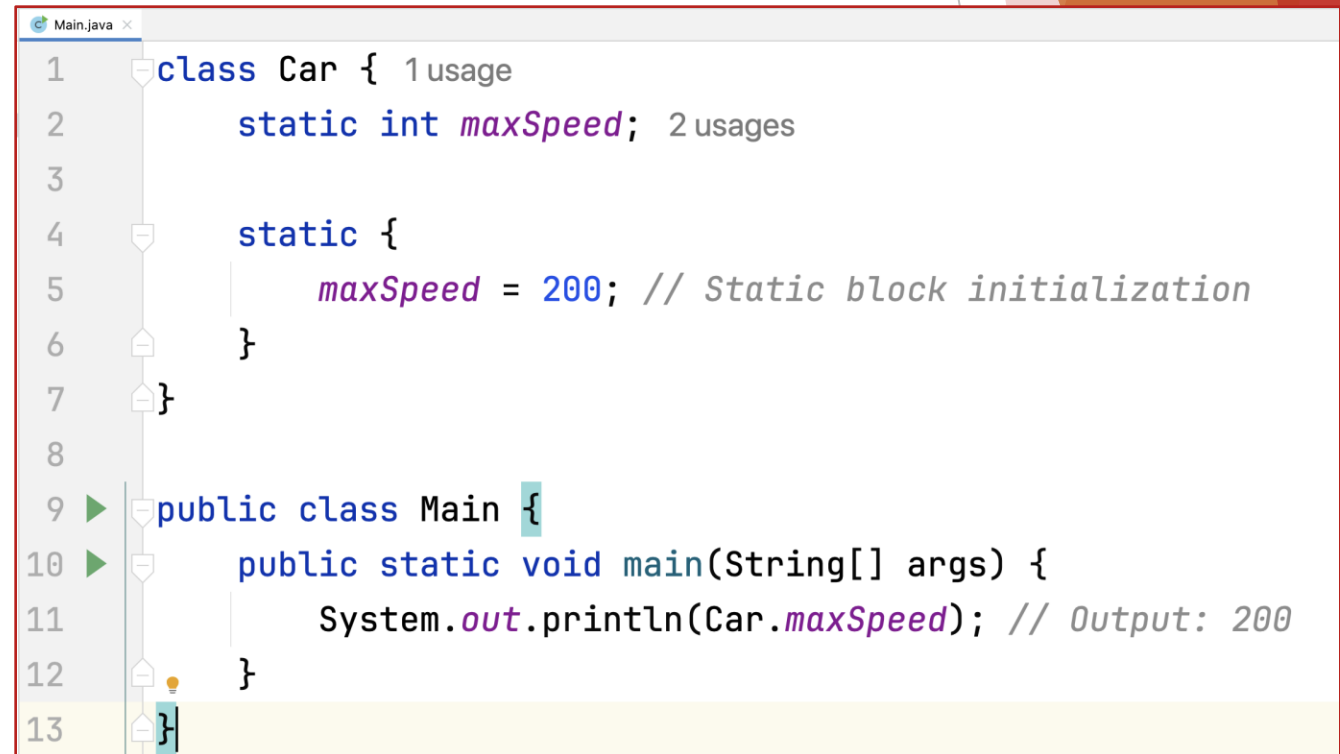
- ▶ A static method can be called without creating an object of the class.
- ▶ It cannot access instance variables or methods directly.



```
1  class Car { 1 usage
2      static void displayInfo() { 1 usage
3          System.out.println("Cars have 4 wheels.");
4      }
5  }
6
7  public class Main {
8      public static void main(String[] args) {
9          Car.displayInfo(); // Output: Cars have 4 wheels.
10     }
11 }
```

Static Block

- ▶ Used to initialize static variables.
- ▶ Executes once when the class is loaded.



```
1 class Car { 1 usage
2     static int maxSpeed; 2 usages
3
4     static {
5         maxSpeed = 200; // Static block initialization
6     }
7 }
8
9 public class Main {
10     public static void main(String[] args) {
11         System.out.println(Car.maxSpeed); // Output: 200
12     }
13 }
```

Instance vs. Static Members

Feature	Instance Members	Static Members
Belongs To	Object	Class
Access	Requires an object	Directly via class
Initialization	Every time an object is created	Only once when class loads

Interactive Activity

- ▶ Write a class Car with:
 - ▶ A static variable totalCars to track the total number of cars created.
 - ▶ A parameterized constructor to increment totalCars when a new object is created.
 - ▶ A static method to display the total number of cars.

What are Wrapper Classes?

- ▶ Wrapper classes are used to convert primitive data types into objects.
- ▶ They are part of the `java.lang` package.
- ▶ Example: `int` can be converted to `Integer`, `char` to `Character`, etc.

Primitive Types and Their Wrapper Classes

Primitive Type	Wrapper Class
int	Integer
char	Character
byte	Byte
short	Short
long	Long
float	Float
double	Double
boolean	Boolean

Why Use Wrapper Classes?

- ▶ **Object Methods:** Provides methods for conversion and comparison.
- ▶ **Collections:** Java collections (e.g., ArrayList) work only with objects.
- ▶ **Autoboxing and Unboxing:** Automatically converts between primitive types and objects.

Autoboxing and Unboxing

- ▶ **Autoboxing:** Automatically converts a primitive to a wrapper class object.
 - ▶ `int num = 5;`
 - ▶ `Integer intObj = num; // Autoboxing`
- ▶ **Unboxing:** Automatically converts a wrapper class object back to a primitive.
 - ▶ `Integer intObj = 5;`
 - ▶ `int num = intObj; // Unboxing`

Wrapper Class Methods

- ▶ **parseInt(), parseDouble():** Converts a string to a primitive type.
 - ▶ `int num = Integer.parseInt("123");`
- ▶ **toString():** Converts the object value to a string.
 - ▶ `Integer intObj = 5;`
 - ▶ `String str = intObj.toString();`

Example Code

```
WrapperExample.java x
1 ▶ public class WrapperExample {
2 ▶   public static void main(String[] args) {
3       // Autoboxing
4       Integer intObj = 10; // Integer object
5       Double doubleObj = 15.5; // Double object
6
7       // Unboxing
8       int num = intObj; // Primitive int
9       double dNum = doubleObj; // Primitive double
10
11      // Wrapper class methods
12      int parsedNum = Integer.parseInt(s: "100");
13      String str = doubleObj.toString();
14
15      System.out.println("Autoboxing: " + intObj);
16      System.out.println("Unboxing: " + num);
17      System.out.println("Parsed number: " + parsedNum);
18      System.out.println("Double as String: " + str);
19   }
20 }
```

What is an ArrayList?

- ▶ ArrayList is part of the Java Collections Framework.
- ▶ It is a resizable array that grows as needed to accommodate new elements.
- ▶ It implements the List interface and allows duplicate elements.
- ▶ Unlike arrays, ArrayLists do not have a fixed size.

Why Use ArrayList?

- ▶ **Dynamic Sizing:** Unlike arrays, ArrayList can grow or shrink dynamically.
- ▶ **Index-Based Access:** Allows fast random access to elements using index positions.
- ▶ **ArrayList Methods:** Provides various methods for adding, removing, and manipulating data.
- ▶ **Flexibility:** Suitable for storing large amounts of data that may change frequently.

Declaration and Initialization

- ▶ Declaring an ArrayList:

- ▶ `ArrayList<Type> listName = new ArrayList<>();`

- ▶ Example:

- ▶ `ArrayList<Integer> numbers = new ArrayList<>();`

- ▶ `ArrayList<String> names = new ArrayList<>();`

- ▶ Initialization with values:

- ▶ `ArrayList<Integer> numbers = new ArrayList<>(Arrays.asList(1, 2, 3, 4));`

Adding Elements to ArrayList

- ▶ **Add elements at the end:**
 - ▶ `list.add(element);`
 - ▶ **Example:**
 - ▶ `numbers.add(10); // Adds 10 at the end`
- ▶ **Add elements at a specific position:**
 - ▶ `list.add(index, element);`
 - ▶ **Example:**
 - ▶ `numbers.add(2, 15); // Adds 15 at index 2`

Accessing Elements

- ▶ **Get element at a specific index:**

- ▶ `list.get(index);`

- ▶ Example:

- ▶ `String name = names.get(0); // Retrieves the first name`

- ▶ **Get the size of the ArrayList:**

- ▶ `list.size();`

- ▶ Example:

- ▶ `int size = numbers.size(); // Returns the number of elements`

Removing Elements

- ▶ **Remove element by index:**

- ▶ `list.remove(index);`

- ▶ Example:

- ▶ `numbers.remove(2);` // Removes element at index 2

- ▶ **Remove specific element by value:**

- ▶ `list.remove(element);`

- ▶ Example:

- ▶ `names.remove("John");` // Removes "John" from the list

- ▶ **Clear all elements:**

- ▶ `list.clear();`

ArrayList Methods Overview

▶ Common Methods in ArrayList:

- ▶ `add(E e)` - Adds element to the end.
- ▶ `get(int index)` - Retrieves element at the specified index.
- ▶ `remove(Object o)` - Removes the first occurrence of specified object.
- ▶ `remove(int index)` - Removes element at specified index.
- ▶ `set(int index, E element)` - Replaces element at specified index.
- ▶ `contains(Object o)` - Checks if the list contains the specified element.
- ▶ `isEmpty()` - Checks if the list is empty.

Example Code:

Basic ArrayList Operations

```
ArrayListExample.java x
1  import java.util.ArrayList;
2
3  public class ArrayListExample {
4      public static void main(String[] args) {
5          // Creating an ArrayList
6          ArrayList<String> names = new ArrayList<>();
7          // Adding elements
8          names.add("Alice");
9          names.add("Bob");
10         names.add("Charlie");
11         // Accessing elements
12         System.out.println("First name: " + names.get(0));
13         // Modifying elements
14         names.set(1, "David"); // Replaces "Bob" with "David"
15         // Removing elements
16         names.remove(0: "Charlie"); // Removes "Charlie"
17         // Displaying ArrayList
18         System.out.println("Updated list: " + names);
19         // Checking size
20         System.out.println("Size of list: " + names.size());
21         // Checking if list contains an element
22         System.out.println("Contains 'Alice': " + names.contains("Alice"));
23     }
24 }
```

Example: ArrayList with Different Types

▶ ArrayList with Integer:

- ▶ `ArrayList<Integer> numbers = new ArrayList<>();`
- ▶ `numbers.add(10);`
- ▶ `numbers.add(20);`
- ▶ `System.out.println(numbers);`

▶ ArrayList with String:

- ▶ `ArrayList<String> cities = new ArrayList<>();`
- ▶ `cities.add("New York");`
- ▶ `cities.add("London");`
- ▶ `cities.add("Tokyo");`
- ▶ `System.out.println(cities);`